

전체 커리큘럼 개요

CURRICULUM OVERVIEW — 교보생명 디지털 자산 플랫폼 · 50세션 · Phase 1 Prototype

모듈	주제	세션	핵심 구현	산출물
M1	전체 아키텍처 프리뷰 + 환경 셋업	S1 ~ S4	아키텍처 워크스루 · 스켈레톤 파악 · Hardhat fork	개발환경
M2	DMZ 이벤트 파이프라인	S5 ~ S12	202 패턴 · Consumer Group · DLQ · Finalized	DMZ 파이프라인
M3A	TX 상태머신	S13, S16~S19	TX 상태머신 · Idempotency · REVERT/TIMEOUT/REORG 복구	TX 상태머신
M4	내부 원장 + 감사 로그	S23 ~ S26	전이 가드 · 먹등기 · SHA-256 체인 · Reconcile	원장+감사로그
M5	비즈니스 로직 레이어	S27 ~ S34	지갑 매핑 · 조건 판단 → 단건 요청 → 벌크 오케스트레이션	비즈니스 로직
M3B	VASP 추상화	S14, S15, S20~S22	IBlockchainAdapter · 멀티체인 전환 · 복구 핸들러 · VASP 이중 채널	VASP 연동
M6	ERC-1155 NFT 컨트랙트 완전 구현	S35 ~ S40	UUPS+AccessControl · tokenId · mintBatch · Sepolia 배포	KyoboNFT.sol
M7	보안 감사 + 업그레이드 운영	S41 ~ S44	Slither HIGH/MEDIUM 제거 · Storage layout · v2 업그레이드	감사 통과 v2
M8	키 거버넌스 + Phase 1 통합	S45 ~ S50	EIP-712 SafeTx · Travel Rule · E2E 통합 · 장애 주입	Phase 1 Prototype

비동기 트랜잭션 상태 관리와 전이 규칙 설계

TxStateMachineService · VALID_TRANSITIONS · Observer Pattern

강의 25분 + 실습 30분 · 1시간

Phase 1 — 인가 VASP

TxStateMachineService

M3 첫 세션

전체 아키텍처 맵

ARCHITECTURE REMINDER — 신뢰 경계로 시스템을 나눈다 · M3는 레이어 3 비즈니스 로직

내부망

레이어 1 — 사용자

Kyobo App

↓ 사용자 이벤트 / API 응답

레이어 2 — 게이트웨이

Webhook
Receiver

Redis Event
Pipeline

IVASP
Adapter

IBlockchain
Adapter

↓ 발행 요청 / 원장 업데이트

레이어 3 — 코어 서비스

비즈니스 로직

데이터 관리

거버넌스

인터넷

레이어 5 — 퍼블릭 블록 체인

이더리움 EVM

↓ Tx Broadcast / 이벤트 감지

레이어 4 — 인가 VASP

인가 VASP

↑ 발행 요청 / Webhook 이벤트 ↓

M3 핵심 구간: 레이어 2 TX 상태머신 — VASP에 TX를 보낸 후 블록체인이 어떤 경로로 실패하는가, 각 상태(REQUESTED→CONFIRMED→FINALIZED)를 어떻게 추적·복구하는가

M2 → M3 연결

WHERE WE CAME FROM · WHERE WE'RE GOING

M2: "이벤트가 도착했을 때 어떻게 안전하게 처리하는가" → M3: "TX를 보냈을 때 블록체인이 어떤 경로로 실패하는가, 각각 어떻게 복구하는가"

M2에서 배운 것

`ConsumerGroupWorker` — At-least-once + 멍등 처리

`IdempotencyGuard` — requestId 기반 중복 차단

`DLQHandler` — 3회 실패 격리

XAUTOCLAIM으로 PEL 채수신

Finalized 블록 체크 후 원장 업데이트

M3에서 연결되는 곳

S22: `pollStaleRequests` 결과를 `ConsumerGroupWorker`가 처리

S16: `submitMintRequest` 에서 UUID requestId 생성 + VASP 재전송 방어

S17: `NonRetryableError` → 즉시 DLQ / `RetryableError` → Backoff

S21~S22: 콜백 누락 시 폴링 채널이 XAUTOCLAIM 연계로 복구

S13: MINED → CONFIRMED 전이 = Finalized 기준

Phase 1 핵심: `handleMined()` 트리거는 **인가 VASP Webhook(NFT_ISSUED)** → `WebhookReceiver` → Redis Streams → Consumer 경로. 직접 체인 폴링 없음.

트랜잭션(Transaction)이란 무엇인가

WARMUP · BLOCKCHAIN TX BASICS

한 줄 정의: 블록체인 네트워크에 "이 상태 변경을 허가한다"고 서명해서 보내는 요청 메시지

일반 DB와의 차이

구분	일반 DB	블록체인 TX
변경 주체	서버 (중앙화)	개인 키 서명자 (탈중앙)
취소 가능	ROLLBACK 가능	불가 — 영구 기록
처리 시점	즉시 (동기)	채굴 후 확정 (비동기)
검증 주체	DB 엔진	네트워크 전체 노드

TX가 담는 핵심 정보

from 보내는 주소 (서명자)	to 받는 주소 / 컨트랙트
value 전송 ETH (0 = 컨트랙트 호출)	data 함수 호출 인코딩 (mint 등)
nonce TX 순번 — 중복·리플레이 방지	gas limit / price 수수료 상한·단가
signature (v, r, s) 개인 키 서명 → 위변조 불가 · 인가 주체 증명	

NFT 발행 시: to = 컨트랙트 주소 / value = 0 / data = `mint(address, tokenId, amount)` 인코딩 / 서명 = 발행 권한 보유 지갑의 개인 키

TX가 블록체인에 도달하는 과정

WARMUP · TX LIFECYCLE — 서명 → mempool → 블록 포함 → 확정



txHash는 영수증 번호가 아니다. 서명 즉시 생성되지만, 블록에 포함되기 전까지는 "처리 예정"일 뿐. txHash가 있어도 실패·드롭될 수 있다.

TX 장애의 두 가지 대표 제약

WARMUP · GAS & NONCE — 원인을 모르면 장애 대응이 불가능하다

블록체인 TX가 mempool에서 드롭되거나 영원히 큐에 묶이는 원인은 대부분 이 두 가지다.

Gas — 연산 수수료

TX 실행에 드는 연산 비용을 ETH로 지불.

수수가 너무 낮으면 채굴자가 외면 → mempool 드롭.

수수료 부족 → TIMEOUT

Limit 초과 → REVERT

다음 슬라이드에서 EIP-1559 구조 상세 →

Nonce — TX 순번

지갑 주소마다 0부터 카운팅되는 전송 일련번호.

순번이 맞지 않으면 이후 TX가 전부 블로킹된다.

순번 gap → 큐 블로킹

중복 전송 방지

다다음 슬라이드에서 Nonce 관리 상세 →

Gas — EIP-1559 수수료 구조

WARMUP · GAS · EIP-1559 — mempool 드롭과 REVERT의 원인

EIP-1559 수수료 계산

필드	의미
<code>baseFee</code>	네트워크가 자동 결정 (소각)
<code>maxPriorityFeePerGas</code>	채굴자에게 지급할 팁
<code>maxFeePerGas</code>	내가 낼 수수료 상한
<code>gasLimit</code>	허용할 최대 연산량

```
// 실제 가스 단가 결정
effectiveGasPrice =
  min(maxFeePerGas,
      baseFee + maxPriorityFeePerGas)
// 실제 청구액
actualFee = gasUsed × effectiveGasPrice
```

채굴자 수령: `effectiveGasPrice - baseFee`

네트워크 소각: `baseFee × gasUsed` · 초과분 환불

Gas 관련 TX 장애 패턴

TIMEOUT (mempool 드롭)

`maxFeePerGas` < `baseFee` → 채굴자가 선택 안 함
→ 장시간 PENDING 후 드롭

REVERT (실행 중 실패)

`gasLimit` 초과 시 실행 중단
→ TX는 블록에 포함되나 상태 변경 없음
→ 수수료는 전액 소모

Gas Bump (S17 복구 전략): PENDING 장기 대기 시
동일 nonce로 더 높은 수수료 TX 재전송 → 기존 TX 대체

Nonce — TX 순번 관리

WARMUP · NONCE — 순번이 틀리면 이후 TX 전체가 멈춘다

Nonce 동작 원리

- 지갑 주소마다 **0부터** 시작하는 전송 카운터
- 블록체인은 `nonce = 현재값 + 1` 인 TX만 채굴
- 같은 nonce → 하나만 처리 (나머지 대체 또는 드롭)

리플레이 공격 방지: 동일 TX를 두 번 브로드캐스트해도 nonce가 이미 소진됐으므로 두 번 실행 안 됨

Nonce 관련 장애 패턴

Nonce Gap — 큐 블로킹

nonce 5번 TX가 실패·드롭된 상태에서 6번 전송
→ 6번은 채굴 불가, **mempool에 영원히 대기**



■ ✓ 처리됨 (0~4) ■ ✗ 드롭 (5) ■ ⌚ 블로킹 (6~8)

복구: nonce 5와 동일한 nonce로

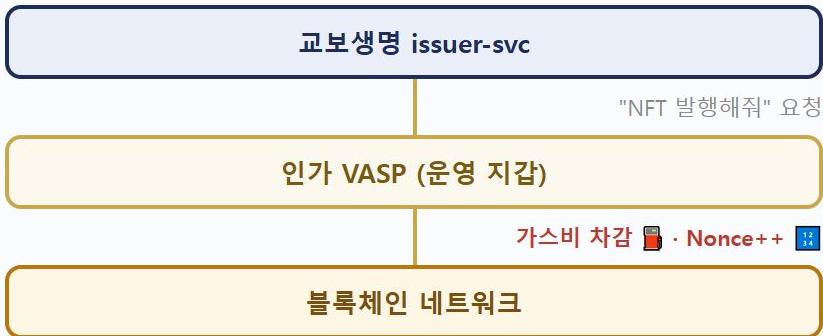
빈 TX (또는 정상 TX)를 높은 수수료로 재전송 → gap 채움

VASP 위탁 — Gas · Nonce 관리도 위탁된다

WARMUP · GAS & NONCE RESPONSIBILITY — TX를 브로드캐스트하는 지갑이 모든 것을 책임진다

VASP의 운영 지갑이 서명·브로드캐스트 → 가스비 납부 + Nonce 카운터 관리 모두 VASP 책임. 교보 issuer-svc는 "발행해줘" 요청만 보낸다.

가스비 흐름



위탁 시 VASP가 처리하는 것

항목	VASP 역할
가스비	운영 지갑에서 선납 → 교보에 수수료 청구
Nonce	운영 지갑의 Nonce 카운터 직접 관리·추적
Gap 복구	Nonce gap 발생 시 재전송·bump 처리
Gas Bump	PENDING 장기 대기 시 수수료 인상 재전송

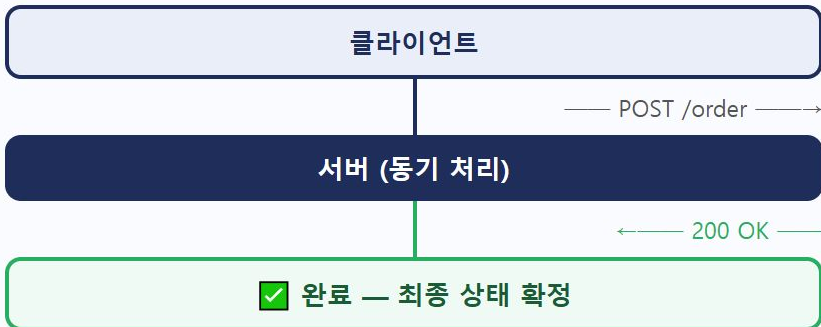
위탁의 핵심 이점: Gas·Nonce 장애 대응을 VASP에 완전 위임 → issuer-svc는 비즈니스 로직에만 집중

단, VASP 수수료에 운영 마진 포함 — 직접 연결 대비 건당 비용은 높을 수 있음

블록체인 TX는 왜 HTTP와 다른가

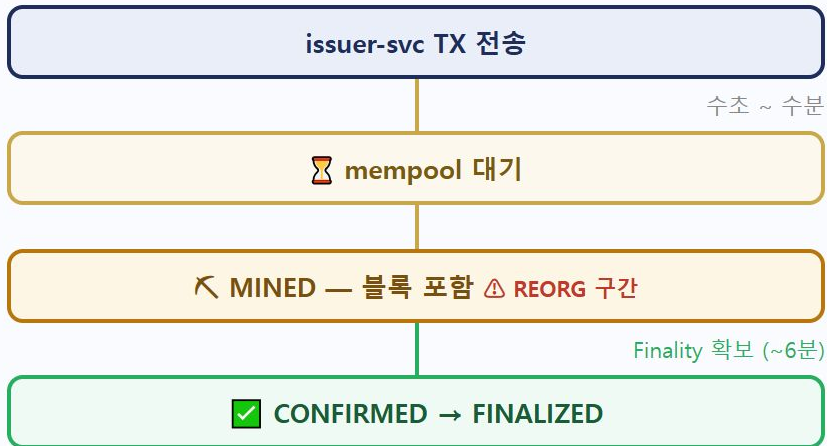
SYNCHRONOUS vs ASYNCHRONOUS

❌ HTTP API — 동기(Synchronous)



- 요청 → 처리 → 응답 한 번에 완료
- 응답을 받으면 그게 최종 상태

✅ 블록체인 TX — 비동기(Asynchronous)



중간 어느 단계에서든 실패 가능: mempool 대기 중 → Gas 부족 드롭(TIMEOUT) | 블록 포함 후 → 컨트랙트 조건 실패(REVERT) | 확정 이후 → 블록 재편으로 TX 소실(REORG)

TX 단계 = 택배 배송 단계

ANALOGY — DELIVERY TRACKING

핵심: 주문 버튼을 눌렀다고 상품이 즉시 도착하지 않는다. 단계별로 추적해야 하고, 각 단계에서의 실패를 **다르게 처리**해야 한다.

TX 상태	택배 단계	의미
REQUESTED	주문 접수	주문은 됐는데 아직 발송 전
SUBMITTED	배송사 접수	택배사에 넘겼는데 운송장 미발행
PENDING	배송 중	운송장 있음, 아직 도착 전
MINED	배달 완료	받긴 했는데 반품 가능 기간 (REORG 가능)
CONFIRMED	수령 확정	반품 기간 지남 → 원장 업데이트 허용
FINALIZED	최종 완료	PoS Finality — 절대 불변 (종단)
FAILED	배달 실패	주소 오류, 수취인 거부 — 중단
REORGED	배달 취소 후 재배달	MINED 후 REORG로 TX 소실, 재처리 대기

왜 상태머신이 필요한가

CRASH RECOVERY SCENARIO

❌ 상태머신 없을 때

t=0 | mint 요청 전송 → VASP

t=1 | VASP → txHash 반환

t=2 | 💣 서버 크래시

...

t=10 | 재시작 — txHash 기록 없음

? 발행됐나 안 됐나?

재시도 → NFT 2개 발행 🚨

포기 → NFT 미발행 🚨

✅ 상태머신 있을 때

t=0 | DB: **REQUESTED** 기록 → VASP 요청

t=1 | DB: **SUBMITTED** + txHash=0xabc

t=2 | 💣 서버 크래시

...

t=10 | 재시작 — DB: SUBMITTED (txHash=0xabc)

VASP에 txHash로 상태 조회
이미 처리됨 확인 → CONFIRMED 전이

✅ 중복 없음

상태머신이 제공하는 것: 어느 시점에 어떤 단계인지 DB에 기록 → 재시작 후에도 "어디까지 처리됐나" 명확히 파악 → 각 단계별 복구 전략 적용 → 중복 실행 방어

"VASP를 쓰면 상태머신이 필요 없지 않나요?"

VASP DELEGATION — VASP의 역할 범위와 한계

VASP는 블록체인 연결 레이어를 대신한다. 하지만 내 서비스의 비즈니스 상태는 여전히 내가 관리해야 한다.

✅ VASP가 해결해주는 것

- › Gas / Nonce 관리 및 최적화
- › TX 서명 및 브로드캐스트
- › mempool 모니터링 · Gas Bump 재시도
- › 노드 인프라 운영

❌ VASP가 해결 못 하는 것

- › Webhook 유실 시 내 서비스 복구
- › 원장 업데이트 타이밍 결정
- › 재시도 시 중복 발행 방지
- › REORG 후 내 원장 롤백

핵심: VASP가 "FINALIZED"를 알려줘도, 그 응답을 받아서 내 상태를 어떻게 전이할지는 내 상태머신의 책임이다.

VASP가 FINALIZED를 줘도 상태머신이 필요한가?

DISCUSSION — 두 가지 실패 시나리오로 답한다

① VASP 응답 자체가 안 올 수 있다

- > **Webhook 유실** — 내 서버 크래시·네트워크 단절 시 콜백 수신 불가
- > **VASP 자체 장애** — 응답 지연·누락 가능

상태머신 없음 → "이 TX 어디 있지?" 알 방법 없음
→ PENDING 영원히 방치 / 재시도 시 중복 발행 🚨

상태머신 있음 → DB에 **SUBMITTED** 기록
→ pollStaleRequests 감지 → VASP 재조회 → 복구 ✅

② CONFIRMED 응답도 반복될 수 있다

- > **REORG 발생** → VASP도 뒤늦게 "TX 소실" 통보
- > CONFIRMED만 믿고 원장 업데이트 → REORG 시 불일치 발생

상태머신 없음 → REORG 통보 받아도
원장 롤백 트리거할 주체가 없다 🚨

상태머신 있음 → **MINED** → **REORGED** 전이
→ 원장 롤백 → 재처리 큐 → 안전한 복구 ✅

결론: CONFIRMED까지는 REORG 가능 — 원장 업데이트 타이밍은 FINALIZED 이후. 그 전이를 제어하는 것이 상태머신의 역할이다.

상태머신(State Machine) 3요소

STATE · TRANSITION · GUARD

● 상태 (State)

지금 어디에 있는가

- > REQUESTED, SUBMITTED
- > PENDING, MINED
- > CONFIRMED, FINALIZED
- > FAILED, REORGED

신호등: 빨강 / 노랑 / 초록

→ 전이 (Transition)

어디로 갈 수 있는가

- > REQUESTED → SUBMITTED
- > MINED → CONFIRMED
- > MINED → REORGED
- > CONFIRMED → FINALIZED

신호등: 빨강→초록, 초록→노랑

🛡 가드 (Guard)

언제 전이가 허용되는가

- > REQUESTED에서 바로
CONFIRMED 불가
- > CONFIRMED → REORGED 불가
(Finality 이후)
- > FINALIZED는 종단 — 이후 전이 없음

신호등: 빨강→노랑 직행 불가

가드는 "잘못된 코드가 데이터를 망치는 것을 막는 안전망"이다. CONFIRMED를 REQUESTED로 되돌리려는 버그가 DB를 오염시키지 못하게 한다.

TX 상태 8개 — 각 상태의 보장

TX STATUS DEFINITIONS · "이 상태에 있을 때 무엇이 참인가"

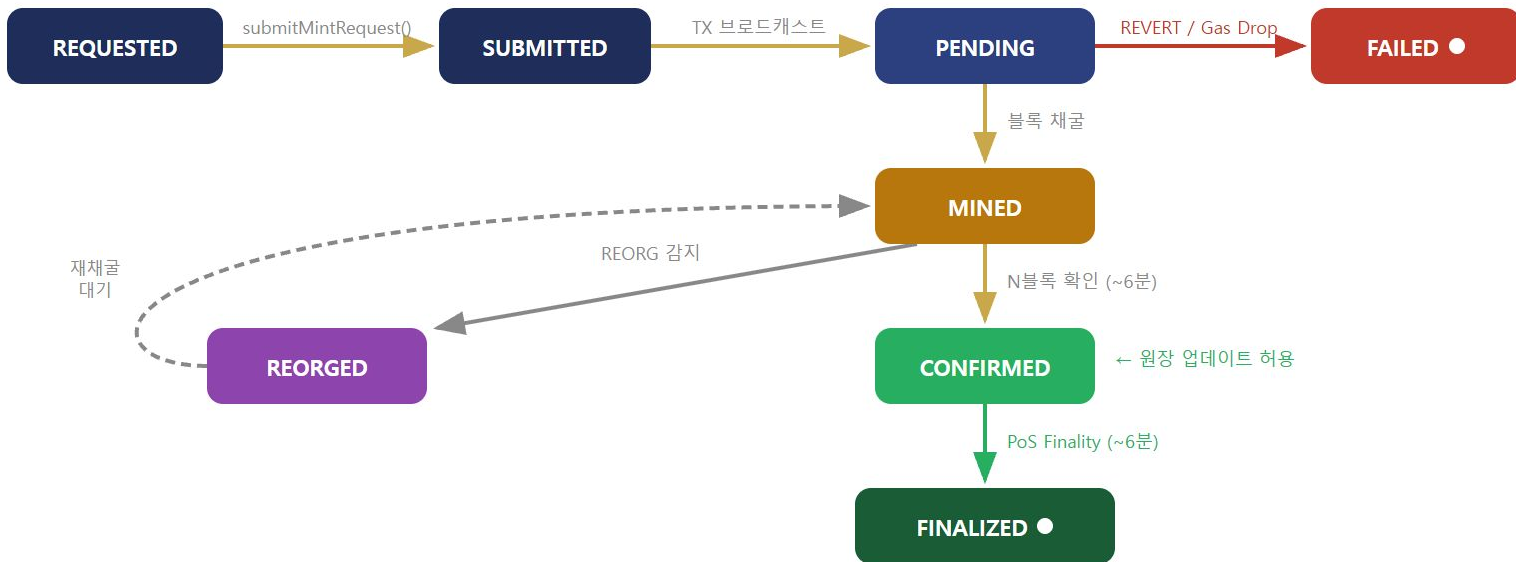
```
export type TxStatus =  
  'REQUESTED' // 요청 생성, VASP 전송 전  
  | 'SUBMITTED' // VASP에 전송됨, TX hash 미획득  
  | 'PENDING' // TX 전송됨, 블록 미채굴  
  | 'MINED' // 블록에 포함됨 (INCLUDED), Finality 미확보  
  | 'CONFIRMED' // 충분한 블록 확인 → 원장 업데이트 허용  
  | 'FINALIZED' // PoS 2/3+ validator 동의 → 절대 불변 — 종단  
  | 'FAILED' // REVERT 또는 최종 실패 — 종단  
  | 'REORGED'; // MINED 후 REORG로 TX 소실, 재처리 대기
```

상태 정의의 원칙: 단순한 "단계"가 아니라 "이 상태에 있을 때 무엇이 참인가"를 기준으로 정의한다.

"PENDING이면 txHash가 있다" · "CONFIRMED면 원장 업데이트 가능" · "FINALIZED는 번복 불가" — 위반 시 버그

상태 전이도

STATE TRANSITION DIAGRAM



진행 중 채굴됨 확정됨 ● 중단 ● 실패 재처리

REORG는 MINED 구간에서만 처리. CONFIRMED 도달 시 충분한 블록이 쌓여 실질적 REORG 위험 없음 → CONFIRMED → REORGED 경로 없음. FINALIZED 이후에는 PoS 합의 규칙상 REORG 자체 불가.

REORGED vs FAILED 구분 기준 — "재시도하면 성공 가능한가?" REORGED: TX는 유효했으나 체인 재편으로 소실 → 재전송하면 성공 가능 | FAILED: REVERT(컨트랙트 거부) · 재시도 횟수 초과 → 같은 내용으로 재시도해도 실패 반복

MINED / CONFIRMED / FINALIZED 구분

FINALITY STAGES · CUSTODY A4/A5/A6 대응

상태	Custody 대응	의미	원장 업데이트
MINED	A4 INCLUDED	블록에 포함됨. REORG 가능 구간. Finality 미확보	❌ 금지
CONFIRMED	A5 CONFIRMED	충분한 블록 확인. 실질적 REORG 위험 없음	✅ 허용
FINALIZED	A6 FINALIZED	PoS 2/3+ validator 동의 (~12분). 번복 절대 불가 — 종단	✅ 완료됨

원장 업데이트는 **CONFIRMED 이후만**. MINED 즉시 원장을 올리면 REORG 시 잔액 오염이 발생한다. 체인에는 NFT 없는데 DB에는 보유 기록 → 이중 발행 혹은 미발행 상태 불일치.

Custody 커리큘럼 Session3 TxAttempt 상태기계(A0~A6)와 동일한 철학 공유. MINED(=A4) / CONFIRMED(=A5) / FINALIZED(=A6) 구분은 두 커리큘럼 모두에서 원장 업데이트 안전 기준으로 사용.

VALID_TRANSITIONS — 왜 전이를 제한하는가

TRANSITION GUARD · SAFETY NET

❌ 전이 제한 없으면

REORG 처리 코드에 버그 발생



CONFIRMED → REQUESTED 되돌리기 시도



DB 오염 — 원장·체인에는 NFT, DB에는 REQUESTED



재처리 로직이 다시 mint 시도 → NFT 2개 발행 🚨

✅ 전이 제한 있으면

CONFIRMED → REQUESTED 전이 시도



VALID_TRANSITIONS 체크: CONFIRMED에서 REQUESTED 없음



InvalidStatusTransitionError 발생



DB 업데이트 차단 — 버그가 데이터 오염으로 이어지지 않음 ✅

전이 제한 = 잘못된 코드가 데이터를 망치는 것을 막는 안전망.

종단 상태(FINALIZED · FAILED)에서 전이 시도 → 즉시 예외 → DB 업데이트 차단 → 재처리 루프 방지

VALID_TRANSITIONS 맵 구현

TxStateMachineService.ts · _transition() 가드

```
// TxStateMachineService.ts - 모듈 레벨 상수
export const VALID_TRANSITIONS: Record<TxStatus, TxStatus[]> = {
  REQUESTED: ['SUBMITTED', 'FAILED'],
  SUBMITTED: ['PENDING', 'FAILED'],
  PENDING: ['MINED', 'FAILED'],
  MINED: ['CONFIRMED', 'REORGED', 'FAILED'], // REORG는 MINED 구간에서만
  CONFIRMED: ['FINALIZED'], // 충분한 블록 → PoS Finality 대기
  FINALIZED: [], FAILED: [], // 종단 - 전이 없음
  REORGED: ['MINED', 'FAILED'],
};

// _transition(): 모든 상태 전이의 단일 진입점
private async _transition(req: MintRequest, to: TxStatus, extra?: Partial<MintRequest>) {
  const from = req.status;
  if (!VALID_TRANSITIONS[from].includes(to))
    throw new InvalidStatusTransitionError(from, to);
  await this.repo.updateStatus(req.id, to, extra);
  const updated = { ...req, status: to, ...extra, updatedAt: new Date() }; // extra 포함
  this.emit('transition', { requestId: req.id, from, to, req: updated });
}
```

MintRequest 데이터 모델

DATA MODEL · 각 필드가 왜 존재하는가

MintRequest 인터페이스

```
export interface MintRequest {  
  // 비즈니스 식별  
  id: string; // Idempotency key  
  userId: string;  
  tokenId: bigint;  
  amount: bigint;  
  // 상태머신  
  status: TxStatus;  
  // TX 추적  
  txHash?: string;  
  blockNumber?: number;  
  // 복구 / 운영  
  retryCount: number;  
  gasPriceGwei?: number;  
  failReason?: string;  
  createdAt: Date;  
  updatedAt: Date;  
}
```

그룹별 역할

비즈니스 식별

id — Idempotency key, VASP 중복 방지
userId/tokenId/amount — 발행 파라미터

상태머신

status — 현재 TX 상태 (TxStatus)

TX 추적

txHash — Gas Bump 대상 식별
blockNumber — REORG 감지 기준점

복구 / 운영

retryCount — 무한 재시도 방지
gasPriceGwei — Gas Bump 이력
failReason — 장애 원인 기록

의존 인터페이스 3종

TxRepository · VaspTxClient · WalletResolver — 생성자 주입

TxRepository — 상태 영속화

```
interface TxRepository {
    save(req): Promise<void>;
    findById(id): Promise<...>;
    updateStatus(
        id, status, extra?
    ): Promise<void>;
    findPendingOlderThan(
        minutes
    ): Promise<...[]>;
}
```

- > extra : txHash, blockNumber, failReason 동시 업데이트
- > findPendingOlderThan : S22 pollStaleRequests 배치 복구용

VaspTxClient — 블록체인 연동

```
interface VaspTxClient {
    submitMint(params: {
        to, tokenId, amount,
        requestId: string // ← Idempotency
    }): Promise<{txHash}>;

    getStatus(txHash):
        Promise<{status, blockNumber,
            revertReason?}>;

    resubmitWithGasBump(
        txHash, gasBumpPercent
    ): Promise<{txHash}>;
}
```

- > requestId : VASP 측 중복 발행 차단
- > 'not_found' : mempool drop → TIMEOUT 처리

WalletResolver — 지갑 주소

```
interface WalletResolver {
    getWalletAddr(
        userId: string
    ): Promise<string>;
}
```

- > userId → 지갑 주소 조회용 추상화
- > 실제 구현은 M5(S27~S29)에서 다룸
- > 지금은 "주소 조회 추상화"로만 이해

M3 전체에서 이 3가지 인터페이스 패턴이 반복된다

DB 먼저 쓰기 원칙

"DB FIRST, EXTERNAL CALL SECOND" · 크래시 복구 가능 여부 결정

❌ 나쁜 순서: VASP 먼저 → DB 나중

issuer-svc

VASP

DB

mint 요청 → TX 전송 중...

OK 수신 ← txHash 반환

❌ 크래시 — DB에 기록 없음

재시작 → DB 조회 → 기록 없음

재시도 → NFT 2개 🚨 | 포기 → 미발행 🚨

✅ 올바른 순서: DB 먼저 → VASP 나중

issuer-svc

DB

VASP

REQUESTED 저장 → REQUESTED 기록됨

mint 요청 →

TX 전송 중...

❌ 크래시

재시작 → DB: REQUESTED 기록 있음

VASP 상태 조회 → 처리됨 확인

✅ SUBMITTED 복귀 — 중복 없음

DB save 후 크래시 → REQUESTED 상태로 기록됨 → pollStaleRequests가 감지 → VASP 재조회 → 이미 전송됐으면 SUBMITTED 복귀 → 전송 전이면 재전송.

이 순서가 At-least-once 환경에서 "처리됐는지 확실히 알 수 없을 때"를 안전하게 만든다.

submitMintRequest — REQUESTED → SUBMITTED

TxStateMachineService.ts:128

```
async submitMintRequest(params: { userId, tokenId, amount }): Promise<string> {
  const id = randomUUID(); // ① Idempotency key
  const req = { id, ...params, status: 'REQUESTED', retryCount: 0,
    createdAt: new Date(), updatedAt: new Date() };
  await this.repo.save(req); // ② DB INSERT 먼저
  try {
    const walletAddr = await this.wallet.getWalletAddr(params.userId);
    const { txHash } = await this.vasp.submitMint(
      { to: walletAddr, tokenId: params.tokenId, amount: params.amount, requestId: id }
    ); // ③ VASP 위탁
    await this._transition(req, 'SUBMITTED', { txHash }); // ④ SUBMITTED 전이
  } catch (err) {
    await this._transition(req, 'FAILED',
      { failReason: `submit failed: ${String(err)}` }); // ⑤ 실패 시 FAILED
    throw err;
  }
  return id;
}
```


핸들러 패턴 — 가드 + 단일 책임

AT-LEAST-ONCE GUARD · return vs throw

핸들러 구현 패턴

```
// Phase 1: 인가 VASP Webhook → Redis Streams → Consumer 경로
async handleMined(requestId, blockNumber): Promise<void> {
  const req = await this._getOrThrow(requestId);
  if (req.status !== 'PENDING' && req.status !== 'SUBMITTED') return; // ← 가드
  await this.repo.updateStatus(requestId, 'MINED', { blockNumber });
}

async handleConfirmed(requestId): Promise<void> {
  const req = await this._getOrThrow(requestId);
  if (req.status !== 'MINED') return; // ← 가드
  await this.repo.updateStatus(requestId, 'CONFIRMED');
}

async handleFailed(requestId, reason): Promise<void> {
  await this.repo.updateStatus(requestId, 'FAILED', { failReason: reason });
}
```

왜 throw가 아닌 return인가

- > Webhook / Redis Consumer는 **At-least-once**
- > 같은 이벤트가 네트워크 오류·재시작으로 두 번 이상 올 수 있음

throw 선택 시: 에러 로그 → 불필요한 알람 → 운영자 대응 낭비

At-least-once 환경에서 "정상적인 중복"을 에러로 취급

return 선택 시: 이미 완료된 상태 → 할 일 없음 → 멍등 (Idempotent) 처리 → 중복 호출이 와도 데이터 변화 없음



이미 CONFIRMED된 TX에 handleMined 재배달 → return → PEL 정상 XACK → 멍등 처리

17

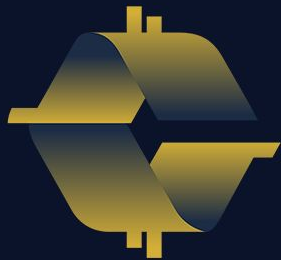
Observer Pattern TxTransitionEvent

상태 전이 발생 시 이벤트 구독

서비스 코드 무변경으로 감사 로그·메트릭·알림 추가

EventEmitter 기반

TxStateMachineService.on('transition')



COINCRAFT
ACADEMY

Observer Pattern — 구조

TxTransitionEvent · EventEmitter extends · _transition()

```
// 이벤트 타입
export interface TxTransitionEvent {
  requestId: string;
  from:      TxStatus;
  to:        TxStatus;
  req:       MintRequest; // 전이 시점 스냅샷 - status.extra.updatedAt 반영
}

// TxStateMachineService는 EventEmitter를 extends
export class TxStateMachineService extends EventEmitter {
  private async _transition(req: MintRequest, to: TxStatus, extra?: Partial<MintRequest>) {
    const from = req.status;
    if (!INVALID_TRANSITIONS[from].includes(to))
      throw new InvalidStatusTransitionError(from, to); // ㉠ 전이 가드
    await this.repo.updateStatus(req.id, to, extra); // ㉡ DB 저장
    const updated = { ...req, status: to, ...extra, updatedAt: new Date() };
    this.emit('transition', { requestId: req.id, from, to, req: updated }); // ㉢ 이벤트 방출
  }
}
```

모든 상태 전이는 `_transition()` 단일 진입점을 통과 — 가드·저장·이벤트가 항상 함께 실행된다

Observer Pattern — 구독자 예시

TxStateMachineService 코드 수정 없이 반응 추가

```
const svc = new TxStateMachineService(repo, vaspClient, walletResolver);

// 1. CONFIRMED → 원장 업데이트 (핵심 비즈니스)
svc.on('transition', async (evt: TxTransitionEvent) => {
  if (evt.to === 'CONFIRMED') await ledger.recordHolding(evt.req);
});

// 2. FAILED → 운영팀 알림
svc.on('transition', async (evt) => {
  if (evt.to === 'FAILED') await alertService.send({ severity: 'HIGH', message: `TX failed: ${evt.req.failReason}` });
});

// 3. 감사 로그 (모든 전이)
svc.on('transition', async (evt) => {
  await auditLog.append({ requestId: evt.requestId, from: evt.from, to: evt.to });
});

// 4. 메트릭
svc.on('transition', (evt) => {
  metrics.increment(`tx.${evt.from}_to_${evt.to}`);
});
```

4가지 구독자 모두 TxStateMachineService 코드 한 줄도 수정 안 함 — 서비스는 "전이가 일어났다"만 알린다

실습 — StudentTxStateMachineService 직접 구현

HANDS-ON · course/exercises/M3/S13_tx_statemachine.ts

구현 목록 (6개 TODO)

#	메서드	핵심 규칙
1	<code>VALID_TRANSITIONS</code>	8개 상태 → 허용 전이 배열 정의
2	<code>_transition()</code>	가드 → <code>repo.updateStatus</code> → <code>emit</code>
3	<code>handleMined/Confirmed/Finalized</code>	상태 가드 — 이미 진행됐으면 <code>return</code>
4	<code>handleTimeout()</code>	gas bump, 상태 유지 → <code>repo</code> 직접 호출
5	<code>handleReorg()</code>	REORGED → <code>wait</code> → <code>getStatus</code> → MINED FAILED
6	Observer 구독자	CONFIRMED 전이 시에만 이벤트 수 집

자동 채점 (npm run exercise:s13)

검증 항목	체크 수
[1] <code>VALID_TRANSITIONS</code> 8개 상태	8개
[2] <code>_transition</code> 가드 + DB 저장	3개
[3] 핸들러 가드 — <code>at-least-once</code>	4개
[4] <code>handleTimeout</code> — PENDING 유지	3개
[5] <code>handleReorg</code> — 재채굴 / 포기	4개
[6] Observer — CONFIRMED 필터	4개

핵심 함정: ① `handleTimeout` 은 `_transition()` 호출 금지 — PENDING→PENDING은 `VALID_TRANSITIONS`에 없음 ② `handleReorg` 재조회 후 업데이트 대상은 `reorgedReq` (상태=REORGED), `req` (MINED) 아님

S13 핵심 요약

KEY TAKEAWAYS

개념	핵심
왜 상태머신	TX는 즉시 확정 안 됨 — 각 단계를 DB에 기록해야 재시작 후 복구 가능
DB 먼저 쓰기	<code>repo.save(REQUESTED)</code> 후 VASP 호출 — 크래시 후 고아 TX 방지
가드 패턴	각 핸들러: <code>if (status !== 허용된_이전) return</code> — At-least-once 중복 방어
CONFIRMED 기준	MINED(=INCLUDED)가 아닌 CONFIRMED 이후만 원장 업데이트 허용 — REORG 오염 방지
REORG 구간	REORG는 MINED 구간에서만 처리 — FINALIZED 이후 REORG 불가 (PoS 보장)
종단 상태	FINALIZED·FAILED 모두 <code>VALID_TRANSITIONS = []</code> — 잘못된 재처리 차단
Observer Pattern	<code>svc.on('transition', handler)</code> — TxStateMachineService 변경 없이 구독자 추가
Custody 연결	MINED=A4 INCLUDED · CONFIRMED=A5 · FINALIZED=A6 — 동일 철학

S16 예고: TX 중복 방어 — `submitMintRequest` 의 UUID requestId를 VASP에도 전달해 At-least-once 환경에서 멍등성을 보장하는 Idempotency 패턴